



GOA ELECTRONICS LIMITED

A Subsidiary of EDC Ltd. (A Government of Goa Undertaking)

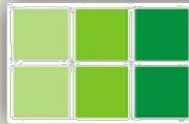
Important: The information contained in this document is for exclusive use of Goa Electronics Limited. The information contained in this document should not be passed on to any third party in part or full.

**REVISION HISTORY**

Revision No./Version No.	Date	Prepared by	Approved by	Details of Change
1.0	17-Feb-2017	Sagar Jaiwar(Assistant Manager)	Revati Mujumdar(CEO)	-
2.0	17-Aug-2017	Anant Yende(CTO)	Revati Mujumdar(CEO)	Additional standards and Revisions
3.0	23-Aug-2024	Varad Harmalkar (Software developer)		.NET coding standards

**TABLE OF CONTENTS**

Sr. No	Title	Page No
1	Introduction	4
2	Indendation	4
3	Formatting	4
4	Code Commenting	4
5	Naming Conventions	5
6	Exceptions	5
7	General	6
8	Encryption standard	7
9	Session Handler	7
10	Validations	7
11	Method guidelines	8



INTRODUCTION

The purpose of this document is to outline the coding standards and best practices for developing applications using ASP.NET. These standards are intended to ensure that our codebase remains clean, maintainable, and consistent across all projects and team members. Adhering to these guidelines will facilitate better collaboration, reduce errors, and improve the overall quality of our software.

1. Indentation

1.1 Proper and consistent indentation is important in producing easy to read and maintainable programs. Indentation should be used to:

- i) Emphasize the body of a control statement such as a loop or a select statement .
- ii) Emphasize the body of a conditional statement .
- iii) Emphasize a new scope block .

1.2 A minimum of 3 spaces shall be used to indent. Generally, indenting by 3 or 4 spaces is considered to be adequate.

2. Formatting

2.1 Never declare more than 1 namespace per file.

2.2 Avoid putting multiple classes in a single file.

2.3 Always use curly braces ({ and }) in conditional statements.

2.4 Declare each variable and initialise with the default value.

2.5 Place namespace "using" statements together at the top of file.

2.6 Write clear and concise comments to explain the purpose of the code

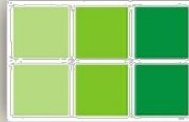
2.7 Prioritize readability over cleverness. Write code that others can understand.

2.8 Don't Repeat Yourself. Reuse code where possible.

3. Code Commenting

3.1 Use // or /// but never /* ... */

Do not "flowerbox" comment blocks.



Example:

```
// *****
```

```
// Comment block
```

```
// *****
```

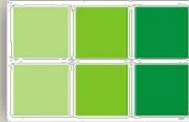
Use inline-comments to explain assumptions, known issues, and algorithm insights.

4.Naming Conventions

- 4.1 Always use Camel Case or Pascal Case names.
- 4.2 Avoid ALL CAPS and all lowercase names. Single lowercase words or letters are acceptable.
- 4.3 Do not use names that begin with a numeric character.
- 4.4 Always choose meaningful and specific names.
- 4.5 Use uppercase for two-letter abbreviations.
- 4.6 Do not use C# reserved words as names.

5.Exceptions

- 5.1 Never declare an empty catch block.
- 5.2 The exception log to be maintained for all the exceptions thrown by the applications
- 5.3 The exception log need to be reviewed every day and corrective action to be taken to avoid repeating the same exception again
- 5.4 The exceptions which are not relevant and are being logged repeatedly to be removed from logging. This should preferably be the custom exceptions written exclusively.
- 5.5 Avoid nesting a try/catch within a catch block.
- 5.6 Only use the finally block to release resources from a try statement.
- 5.7 Always use validation to avoid exceptions.
- 5.8 Do not log binary or byte data.



Example:

```
// Bad!

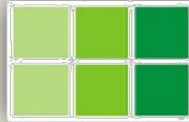
try
{
    conn.Close();
}
Catch(Exception ex)
{
    // handle exception if already closed!
}

// Good!

if(conn.State != ConnectionState.Closed)
{
    conn.Close();
}
```

6.General

- 6.1 Object Initialization and Closing : Initiate object and close object after use explicitly.
- 6.2 Tab Indexing of the controls : Ensure the controls are placed in correct tab sequence so that user can access the controls with keyboard.
- 6.3 Maintaining of Tab Sequence on Post Back
- 6.4 Validation Summary : Always display validation summary.
- 6.5 Proper relevant Validation Messages to be displayed.
- 6.6 Add Shortcut Keys for faster data entry
- 6.7 Data Validation before Save. Validate data at client and server end before processing/saving.
- 6.8 Disable control on same action to limit resubmits.
- 6.9 Don't use any third party dll/ tools without confirmation from the CTO sir.
- 6.10 Add default button for the section to help user to submit form on click of enter button on keyboard.



7. Encryption standards

- 7.1 All the data being moved across pages using query string /post method to be encrypted
- 7.2 AES encryption to be used for cases where the decryption is applicable
- 7.3 The password should be MD5/SHA encrypted and must be combined with a SALT and the encryption must be done at client side when receiving a login request
- 7.4 The input received and output given from a webservice must be AES encryption with the secret key shared with the receiver
- 7.5 Preferably different encryption keys to be used for different consumers of the services to avoid misuse of others data.

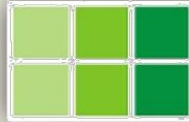
8. Session handler

- 8.1 All session objects to be disposed on logout or time-out function and on logon page load.
- 8.2 Avoid using session object for the local data in the form.

9. Validations

Following fields to be validated before submitting to the server:

- 9.1 Date to be validated with server date for comparisons
- 9.2 Date format to be ensured to prevent any data loss during conversion.
- 9.3 The search or input pages on public domain should have a captcha code implementation.
- 9.4 Validation summary to be compulsorily displayed in pop-up and in form footer
- 9.5 Confirm alert to be shown for following critical actions
Save ,Update ,Delete, Cancel, Back, etc
- 9.6 Back or cancel button should intimate user about loss of edited information if proceeded with cancel/back
- 9.7 After action button is clicked the action button should be disabled or hidden to prevent user from submitting the same action multiple times



- 9.8 After the response is received from the server the same need to be presented to the user before navigating to a new page unless otherwise specified by client for auto redirection.
- 9.9 Proper success or error panel to be shown with specific message to alert user about the processing outcome.
- 9.10 Every control should have the field length as per the data structure definition.
- 9.11 Control height and width to be as per the type of the data being captured or as per the UI/UX theme of the project and same uniformity to be ensured across complete Project Avoid using unnecessary post backs.

10. Method Guidelines

10.1 Method Length:

Keep methods short and focused. Ideally, a method should not exceed 30 lines of code. If a method becomes too long, refactor it into smaller methods and each method does a definitive purpose.

10.2 Parameter Count:

Avoid methods with too many parameters. If a method requires more than three or four parameters, consider encapsulating the parameters into a class or struct.

10.3 Method Overloading:

Use method overloading judiciously to provide multiple ways to call a method with different parameters. Ensure that the method signatures are distinct and clear and codes are not repeated.

Also Read

- General coding standards and best practices.
- Web developments standards.